Concorrência no MySQL - Atividades

1. Análise Teórica de Anomalias de Concorrência

Cenário: Um operador de caixa do e-commerce iniciou um relatório de fechamento de caixa e executou uma consulta que retornou 50 pedidos ativos (status = 'open') na tabela orders. Enquanto ele analisava os números (com a transação ainda aberta no banco), outro processo confirmou o pagamento de 2 pedidos e inseriu 3 novos pedidos na mesma tabela. Ao reexecutar a mesma consulta dentro da mesma transação, o operador obteve um resultado diferente.

- **Questão A:** Quais das anomalias clássicas de concorrência (*Dirty Read*, *Non-Repeatable Read* ou *Phantom Read*) ocorreram ou poderiam ocorrer nesse cenário? Explique cada uma com suas palavras.
- **Questão B:** Qual é o nível de isolamento padrão do engine InnoDB no MySQL 9 e como o recurso **MVCC** (*Multi-Version Concurrency Control*) impede que a consulta do operador seja afetada por escritas simultâneas de outros usuários?

2. Simulação Prática de Trava de Linha (*Row Lock Wait*) no MySQL

Objetivo: Vivenciar o comportamento do SGBD quando duas conexões tentam manipular a mesma linha concorrentemente.

Instruções: Abra duas abas ou janelas de terminal/phpMyAdmin apontando para o banco ecommerce:

1. Na Sessão A (Aba 1), inicie uma transação e aplique um bloqueio exclusivo (Pessimistic Lock) no cliente id = 1 da tabela customers:

```
START TRANSACTION;
SELECT * FROM customers WHERE id = 1 FOR UPDATE;
```

2. Na Sessão B (Aba 2), inicie uma transação paralela e tente atualizar o telefone do mesmo cliente (id = 1):

```
START TRANSACTION;
UPDATE customers SET phone = '42999999999' WHERE id = 1;
```

3. Responda:
    a. O que acontece com a consulta executada na Sessão B? Ela finalizou imediatamente ou ficou congelada?
    b. Execute o comando COMMIT; na Sessão A. O que acontece imediatamente na Sessão B?

3. Testando Diferenças entre READ COMMITTED e REPEATABLE READ

Objetivo: Comparar como a mudança do nível de isolamento altera a visibilidade de alterações confirmadas por outras transações.

Instruções:

1. Na Sessão A, altere o nível de isolamento da sessão e inicie a transação:

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
START TRANSACTION;
SELECT stock FROM products WHERE id = 1;
```

2. Na Sessão B, faça a atualização do estoque do produto id = 1 na tabela products e confirme imediatamente:

```
START TRANSACTION;
UPDATE products SET stock = stock - 1 WHERE id = 1;
COMMIT;
```

3. Volte para a Sessão A (sem dar commit) e repita a consulta:

```
SELECT stock FROM products WHERE id = 1;
```

4. Questão: O valor do estoque retornado na Sessão A mudou? O que aconteceria se a Sessão A estivesse utilizando o nível padrão REPEATABLE READ?